STUDENT NAME: AMANDEEP	MODULE: My Practice	CODE: SPRING_001_BOOT
BATCH: ABES_JAS26_FSD JAVA_IIIYear_Batch 02	COURSE: Working with Spring Framework with AI	STATUS: In Progress

Practice Worksheet – Working with Spring Framework with AI

Master enterprise Spring Boot architecture: Inversion of Control (IoC), Bean Lifecycle constraints, Spring JDBC repositories, Spring Boot REST CRUD APIs, and Spring Data JPA relationships.

KEY LEARNING OBJECTIVES:

- Configure Spring IoC container with Java Configuration and Dependency Injection.
- Implement transactional database operations using Spring JDBC and RowMapper.
- Architect production-grade RESTful CRUD APIs with Jakarta Validation and global exception handling.
- Model bidirectional One-to-Many relational domain entities with Spring Data JPA.

STEP-BY-STEP WORKSHEET INSTRUCTIONS:

Step 1: Analyze enterprise business requirements and design the layered architecture.
Step 2: Annotate classes with standard Spring stereotypes (@Configuration, @Service, @Repository, @RestController).
Step 3: Implement clean DTOs, repository interfaces, and transactional service methods.
Step 4: Document REST endpoint mappings, HTTP status codes, and test curl requests.

EXERCISE 6.1: SPRING IOC, BEAN FACTORY & CONSTRAINT INJECTION

Problem Statement: Configure a Spring application using Java-based configuration (`@Configuration`). Define a bean `DiscountService` with properties `discountPercentage` and `maxDiscountAmount` injected via `@Value` from `application.properties`. Demonstrate constructor-based dependency injection into a `CheckoutService` bean using `@Autowired` and `@Qualifier`.

Reference Baseline / Code / Scenario:

```
@Configuration public class AppConfig { @Bean public DiscountService discountService() { return new DiscountService(); } }
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] @Configuration & @Bean Class: _____

[2] Constructor Dependency Injection: _____

[3] @Value Property Binding: _____

EXERCISE 6.2: SPRING JDBC USER SEARCH REPOSITORY & ROWMAPPER

Problem Statement: Create a `UserRepository` using `JdbcTemplate`. Implement `public Optional findByEmail(String email)` executing a parameterized SQL query `SELECT id, name, email, role FROM users WHERE email = ?`. Implement a custom `RowMapper` lambda to map result set rows to the `User` domain entity.

Reference Baseline / Code / Scenario:

```
return jdbcTemplate.queryForObject(sql, (rs, rowNum) -> new User(rs.getLong("id"), ...), email);
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] JdbcTemplate Parameterized Query: _____

[2] Custom RowMapper Implementation: _____

[3] Optional Empty / Error Handling: _____

EXERCISE 6.3: SPRING BOOT PRODUCTION RESTFUL CRUD API DESIGN

Problem Statement: Develop a `ProductRestController` exposing CRUD endpoints for manufactured goods: `GET /api/products`, `POST /api/products` (with `@Valid ProductRequestDTO`), and `DELETE /api/products/{id}`. Enforce standard HTTP status codes (`200 OK`, `201 Created`, `204 No Content`, `404 Not Found`) and `@RestControllerAdvice` error responses.

Reference Baseline / Code / Scenario:

```
@PostMapping public ResponseEntity createProduct(@Valid @RequestBody ProductRequestDTO dto) { ... }
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] RestController Mapping & Verbs: _____

[2] DTO Validation Annotations: _____

[3] @RestControllerAdvice Global Handler: _____

EXERCISE 6.4: SPRING DATA JPA ONE-TO-MANY TEAM AND PLAYER MAPPING

Problem Statement: Model a bidirectional One-to-Many relationship between `Team` and `Player` using JPA annotations (`@OneToMany(mappedBy = "team", cascade = CascadeType.ALL, fetch = FetchType.LAZY)` and `@ManyToOne @JoinColumn(name = "team_id")`). Write a `PlayerRepository` extending `JpaRepository` with a derived JPQL query `findByTeamName(String teamName)`.

Reference Baseline / Code / Scenario:

```
@OneToMany(mappedBy = "team", cascade = CascadeType.ALL, orphanRemoval = true) private List players = new ArrayList<>();
```

STUDENT SOLUTION & CODE WORKSPACE:

[1] Team & Player Entity Annotations: _____

[2] Cascade & Lazy Fetch Policies: _____

[3] Spring Data JpaRepository Query: _____

SELF-REVIEW & VERIFICATION CHECKLIST:

- [ ] **IoC Best Practices:** Dependency injection uses constructor injection over field injection.
- [ ] **Data Access Security:** SQL queries strictly parameterized with zero string concatenation.
- [ ] **REST Conventions:** Endpoints follow standard resource naming and proper HTTP status codes.
- [ ] **JPA Integrity:** Bidirectional relationships maintain helper methods to keep both sides in sync.
- [ ] **Exception Governance:** Global @RestControllerAdvice captures ResourceNotFound and validation errors.

CRITICAL REFLECTION & LEARNING SYNTHESIS:

- *Why is constructor injection strongly recommended over field injection (`@Autowired` on private fields)?*
- *How does `FetchType.LAZY` prevent the catastrophic N+1 query problem in relational JPA applications?*

Reflection Notes:

_____

_____

COMPLETION STATUS:	EVALUATOR & MENTOR REMARKS:
[ ] COMPLETED (Meets All Solution Criteria) [ ] NEEDS REVISION (Resubmit Corrected Work) **Evaluator Signature:** _____	Feedback Notes: _____ Rubric Evaluation: [ ] Correctness [ ] Code Quality [ ] Best Practices